

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №9
по дисциплине «Вычислительная математика»
Тема: Составные формулы прямоугольников, трапеций, Симпсона.

Студент гр. 4342

Садыков С.Р.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Цель лабораторной работы — изучить численное вычисление определённого интеграла с помощью квадратурных формул прямоугольников, трапеций и Симпсона. В ходе работы требуется реализовать вычисление заданной подынтегральной функции и программные функции для нахождения интеграла каждым из указанных методов. Также необходимо применить правило Рунге для оценки погрешности и подобрать наименьшее значение n , то есть наибольшее значение шага h , при котором приближённое значение интеграла будет найдено с заданной точностью ε . Дополнительно требуется сравнить работу формул прямоугольников, трапеций и Симпсона по полученным значениям интеграла, числу разбиений n и точности результата.

Задание.

Повышения точности численного интегрирования добиваются путем применения составных формул. Для этого при нахождении определенного интеграла отрезок $[a, b]$ разбивают на четное $n = 2m$ число отрезков длины $h = (b - a) / n$ и на каждом из отрезков длины $2h$ применяют соответствующую формулу. Таким образом получают составные формулы прямоугольников, трапеций и Симпсона.

На сетке $x_i = a + ih$, $y = f(x_i)$, $i = 0, 1, 2, \dots, 2m$, составные формулы имеют следующий вид:

Формула прямоугольников

$$\int[a; b] f(x) dx = h * \sum_{i=0}^{n-1} f(x_i + h/2) + R_1;$$

Формула трапеций

$$\int[a; b] f(x) dx = (h/2) * \sum_{i=0}^{n-1} (f(x_i) + f(x_{i+1})) + R_2;$$

Формула Симпсона

$$\int[a; b] f(x) dx = (h/3) * \sum_{i=0}^{m-1} (f(x_{2i}) + 4 * f(x_{2i+1}) + f(x_{2i+2})) + R_3,$$

где R_1, R_2, R_3 — остаточные члены. При $n \rightarrow \infty$ приближенные значения интегралов для всех трех формул, в предположении отсутствия погрешностей округления, стремятся к точному значению интеграла [1, 7, 8].

Для практической оценки погрешности квадратурной формулы можно использовать правило Рунге. Для этого проводят вычисления на сетках с шагом h и $h/2$, получают приближенные значения интеграла I_h и $I_{h/2}$ и за окончательные значения интеграла принимают величины:

$$I = I_{h/2} + (I_{h/2} - I_h) / 3 \text{ — для формулы прямоугольников;}$$

$$I = I_{h/2} + (I_{h/2} - I_h) / 3 \text{ — для формулы трапеций;}$$

$$I = I_{h/2} + (I_{h/2} - I_h) / 15 \text{ — для формулы Симпсона.}$$

За погрешность приближенного значения интеграла для формул прямоугольников и трапеций тогда принимают величину: $|I_{h/2} - I_h| / 3$,

а для формулы Симпсона: $|I_{h/2} - I_h| / 15$.

В лабораторной работе №6 требуется, используя квадратурные формулы прямоугольников, трапеций и Симпсона, вычислить значения заданного интеграла и, применив правило Рунге, найти наименьшее значение n , то есть наибольшее значение шага h , при котором каждая из указанных формул дает приближенное значение интеграла с погрешностью ε , не превышающей заданную.

Порядок выполнения лабораторной работы №6.

- 1) Составить программы-функции для вычисления интегралов по формулам прямоугольников, трапеций и Симпсона.
- 2) Составить программу-функцию для вычисления подынтегральной функции.
- 3) Составить главную программу, содержащую оценку по Рунге погрешности каждой из перечисленных выше квадратурных формул, удваивающих n до тех пор, пока погрешность не станет меньше ε , и осуществляющих печать результатов: значения интеграла и значения n для каждой формулы.
- 4) Провести вычисления по программе, добиваясь, чтобы результат удовлетворял требуемой точности.
- 5) Результаты работы оформить в виде краткого отчета, содержащего сравнительную оценку применяемых для вычисления формул.

Варианты заданий приведены в таблице 4.1. $\varepsilon = 0.01; 0.001; 0.0001$.

Вариант 17: $f(x) = \int_0^1 (x/(1+x^2))e^{-x^2} dx$

Выполнение работы.

1. Составить программы-функции для вычисления интегралов по формулам прямоугольников, трапеций и Симпсона.

На первом этапе выполнения лабораторной работы была реализована отдельная программа-функция для вычисления значения подынтегрального выражения согласно семнадцатому варианту. Математическая модель функции представляет собой произведение отношения x к сумме единицы и x в квадрате на экспоненту от минус x в квадрате. Интегрирование данной функции проводилось на заданном в варианте отрезке от нуля до единицы. Разделение кода на независимые блоки позволило использовать одну и ту же математическую модель для всех трех методов численного интегрирования.

Для решения задачи были разработаны три функции. В методе прямоугольников использована составная формула средних прямоугольников, где значение функции вычисляется в центральной точке каждого элементарного отрезка. Это позволяет заменить площадь под кривой суммой площадей прямоугольников. В методе трапеций график функции на каждом шаге заменяется прямой линией, соединяющей значения в узлах сетки. При этом значения функции на границах всего интервала интегрирования учитываются с половинным весом, а внутренние узлы суммируются полностью. В методе Симпсона применяется аппроксимация параболами на парах соседних отрезков, что требует обязательного четного количества разбиений. В программе реализовано условие автоматического приведения числа отрезков к четному значению, а суммирование внутренних точек производится с чередующимися коэффициентами четыре и два.

Основная часть программы реализует алгоритм автоматического достижения заданной точности по правилу Рунге. Процесс вычислений начинается с разбиения отрезка на две части и продолжается путем последовательного удвоения числа отрезков. На каждом этапе сравниваются результаты, полученные на текущей сетке и на сетке с вдвое меньшим шагом.

Для оценки фактической погрешности методов прямоугольников и трапеций разность полученных значений делится на три, а для метода Симпсона на пятнадцать.

Расчеты проводились для трех уровней точности, указанных в задании: одна сотая, одна тысячная и одна десятитысячная. После выполнения условия остановки по достижении требуемой погрешности, программа вычисляет окончательное уточненное значение интеграла с учетом найденной поправки Рунге. Сравнительный анализ результатов показал, что метод Симпсона достигает заданной точности при значительно меньшем количестве разбиений интервала, чем методы прямоугольников и трапеций, что подтверждает его высокую вычислительную эффективность для решения задач данного типа.

2. Составить программу-функцию для вычисления подынтегральной функции

На втором этапе выполнения лабораторной работы была составлена отдельная программа-функция для вычисления значения подынтегрального выражения. По условию семнадцатого варианта требуется вычислить определенный интеграл от нуля до единицы для функции, представляющей собой отношение x к сумме единицы и x в квадрате, умноженное на экспоненту в степени минус x в квадрате.

Следовательно, подынтегральная функция имеет вид произведения дробного выражения на экспоненциальную часть. Для вычисления этой зависимости в программном коде была реализована отдельная функция, которая принимает на вход значение аргумента x и возвращает результат вычисления данного математического выражения. Процесс расчета внутри функции включает возведение аргумента в квадрат, нахождение суммы в знаменателе, деление x на полученный результат и последующее умножение на значение экспоненты от отрицательного квадрата x .

Поскольку интегрирование выполняется на отрезке от нуля до единицы, функция определена на всем промежутке корректно. Знаменатель дроби всегда больше нуля, а экспонента определена для любых вещественных чисел, что гарантирует отсутствие ошибок вычисления и непрерывность функции на заданном интервале.

Выделение подынтегральной функции в отдельный программный блок делает структуру приложения более логичной и универсальной. Все основные алгоритмы численного интегрирования обращаются к одной и той же функции. Это обеспечивает единообразие расчетов и позволяет при необходимости легко изменить интегрируемое выражение в одном месте, не затрагивая программную реализацию самих методов прямоугольников, трапеций и Симпсона.

3. Составить главную программу, содержащую оценку по Рунге погрешности каждой из перечисленных выше квадратурных формул, удваивающих n до тех пор, пока погрешность не станет меньше ε , и осуществляющих печать результатов: значения интеграла и значения n для каждой формулы

На третьем этапе была составлена головная программа, которая объединяет ранее реализованные функции численного интегрирования и выполняет автоматический подбор количества разбиений для каждой квадратурной формулы. Основная задача этой части программы заключается в том, чтобы определить минимальное значение количества отрезков, при котором оценка фактической погрешности становится меньше заданной точности.

В головной программе задаются пределы интегрирования для семнадцатого варианта: нижний предел равен нулю, верхний предел равен единице. Также задаются три значения требуемой точности: одна сотая, одна тысячная и одна десятитысячная. Для каждого из этих уровней точности программа последовательно выполняет вычисления по формуле средних прямоугольников, формуле трапеций и формуле Симпсона.

Проверка точности выполняется с помощью правила Рунге. Процесс начинается с минимального числа разбиений, равного двум. Сначала интеграл вычисляется на текущей сетке, затем вычисление повторяется на более мелкой сетке, где количество отрезков увеличено в два раза. После этого проводится сравнение двух полученных приближенных значений интеграла. Для формул прямоугольников и трапеций оценка погрешности вычисляется как абсолютная разность результатов на двух сетках, деленная на три. Для формулы Симпсона разность делится на пятнадцать.

Различие в используемых знаменателях обусловлено разным порядком точности методов. Формулы прямоугольников и трапеций имеют второй порядок точности, поэтому в правиле Рунге используется делитель три. Формула Симпсона обладает четвертым порядком точности, что требует использования делителя пятнадцать. Если вычисленная оценка погрешности превышает заданное значение точности, программа производит очередное удвоение количества разбиений и повторяет весь цикл расчетов.

После достижения требуемого условия программа фиксирует найденное количество отрезков разбиения, соответствующее приближенное значение интеграла и расчетную оценку погрешности. Кроме того, на основе полученных данных программа вычисляет окончательное уточненное значение интеграла по формулам экстраполяции, приведенным в методических указаниях. Головная программа выводит на экран итоговую таблицу, содержащую название метода, заданную точность, итоговое количество разбиений, вычисленное значение интеграла, его уточненный вариант и фактическую погрешность по Рунге. Это позволяет наглядно сравнить методы между собой и сделать вывод о том, какой из них является наиболее эффективным для данной задачи.

4. Провести вычисления по программе, добиваясь, чтобы результат удовлетворял требуемой точности

На четвертом этапе были проведены вычисления по составленной программе. Для заданного интеграла были использованы три квадратурные формулы: формула прямоугольников, формула трапеций и формула Симпсона.

Вычисления проводились для трех значений точности: $\varepsilon = 0.01$, $\varepsilon = 0.001$, $\varepsilon = 0.0001$.

Для каждой формулы программа начинала вычисления с малого количества разбиений n , затем последовательно удваивала это значение. На каждом шаге интеграл вычислялся дважды: сначала при текущем значении n , затем при удвоенном количестве разбиений. После этого по правилу Рунге оценивалась погрешность результата.

Если оценка погрешности была больше заданного значения ε , количество разбиений снова увеличивалось в два раза. Процесс продолжался до тех пор, пока оценка погрешности не становилась меньше или равной заданной точности.

В результате были получены следующие значения:

ε	Метод	n	h	Значение интеграла	Оценка погрешности
0.01	Прямоугольник и	4	0.25	0.23534390	0.00397
0.01	Трапеции	4	0.25	0.22450607	0.00759
0.01	Симпсон	4	0.25	0.23209307	0.000416
0.001	Прямоугольник и	8	0.125	0.23260585	0.000913
0.001	Трапеции	16	0.0625	0.23126542	0.000447
0.001	Симпсон	4	0.25	0.23209307	0.000416
0.000	Прямоугольник	32	0.03125	0.23176667	0.0000557

1	и				
0.0001	Трапеции	64	0.015625	0.23168317	0.0000278
0.0001	Симпсон	8	0.125	0.23173129	0.0000241

По результатам вычислений видно, что все три метода позволяют получить приближенное значение интеграла с заданной точностью. При этом формула Симпсона достигает требуемой точности уже при меньшем количестве разбиений. Это объясняется тем, что формула Симпсона имеет более высокий порядок точности по сравнению с формулами прямоугольников и трапеций.

Для наиболее строгой точности $\varepsilon = 0.0001$ формулы прямоугольников и трапеций потребовали $n = 32$ и 64 соответственно, а формула Симпсона уже при $n = 8$ дала погрешность меньше заданной. Следовательно, в данной задаче формула Симпсона оказалась наиболее эффективной.

5. Результаты работы оформить в виде краткого отчета, содержащего сравнительную оценку применяемых для вычисления формул.

В ходе выполнения лабораторной работы было проведено численное вычисление интеграла с использованием трех составных квадратурных формул: формулы прямоугольников, формулы трапеций и формулы Симпсона. Для оценки точности результатов применялось правило Рунге. Расчеты выполнялись для трех значений требуемой точности: $\text{eps} = 0.01$, $\text{eps} = 0.001$ и $\text{eps} = 0.0001$.

По полученным данным видно, что все три метода позволяют найти значение интеграла с заданной точностью, однако эффективность методов различается. Главным критерием сравнения является количество разбиений n , которое потребовалось для достижения заданной погрешности. Чем меньше

значение n , тем меньше вычислений нужно выполнить программе, следовательно, тем эффективнее метод.

Для точности $\text{eps} = 0.01$ все три метода достигли нужной точности уже при $n = 4$. Однако оценки погрешности у методов заметно отличаются. У формулы прямоугольников погрешность составила 0.00397, у формулы трапеций — 0.00759, а у формулы Симпсона — всего 0.000416. Это показывает, что даже при одинаковом числе разбиений формула Симпсона дает значительно более точный результат.

Для точности $\text{eps} = 0.001$ различие между методами становится более заметным. Формуле прямоугольников и формуле трапеций потребовалось увеличить число разбиений до $n = 8$ и 16 соответственно, то есть выполнить два/три удвоения n . При этом формула Симпсона снова достигла требуемой точности уже при $n = 4$. Оценка погрешности Симпсона осталась равной 0.000025159569, что намного меньше заданного значения 0.001.

Для наиболее строгой точности $\text{eps} = 0.0001$ формулы прямоугольников и трапеций потребовали уже $n = 32$ и 64. Это означает, что программе пришлось выполнить четыре/пять удвоения(й) количества разбиений. Формула Симпсона при этом удовлетворила требуемой точности при $n = 8$. Это является главным подтверждением того, что формула Симпсона в данной задаче оказалась наиболее эффективной.

Сравнение значений n можно представить следующим образом:

Точность eps	Прямоугольники	Трапеции	Симпсон
0.01	$n = 4$	$n = 4$	$n = 4$
0.001	$n = 8$	$n = 16$	$n = 4$
0.0001	$n = 32$	$n = 64$	$n = 8$

Из таблицы видно, что при ужесточении требования к точности методы прямоугольников и трапеций требуют значительного увеличения числа разбиений. При переходе от $\text{eps} = 0.01$ к $\text{eps} = 0.0001$ количество разбиений для

этих методов выросло с 4 до 32. У формулы Симпсона число разбиений во всех трех случаях осталось равным 4.

Также можно сравнить оценки погрешности при одинаковом значении $n = 4$ для точности $\text{eps} = 0.01$:

Метод	n	Оценка погрешности
Прямоугольники	4	0.00397
Трапеции	4	0.00759
Симпсон	4	0.000416

По этим данным видно, что формула Симпсона дает ошибку примерно в десятки раз меньше, чем формула прямоугольников и формула трапеций. Это связано с тем, что формула Симпсона имеет более высокий порядок точности. Формулы прямоугольников и трапеций имеют второй порядок точности, а формула Симпсона — четвертый порядок точности. Поэтому при уменьшении шага сетки результат по формуле Симпсона уточняется быстрее.

Интересно также сравнить сами приближенные значения интеграла. При самой строгой точности $\text{eps} = 0.0001$ были получены следующие уточненные значения по Рунге:

Метод	I с уточнением по Рунге
Прямоугольники	0.23176667
Трапеции	0.23168317
Симпсон	0.23173129

Значения, полученные по формулам прямоугольников и трапеций при $n = 32$ и 64 , практически совпадают между собой. Это говорит о том, что оба метода при достаточно мелком разбиении сходятся к одному и тому же значению интеграла. Формула Симпсона уже при малом числе разбиений дает значение, близкое к этим результатам, причем ее оценка погрешности также удовлетворяет заданному условию.

Таким образом, по результатам вычислений можно сделать вывод, что все рассмотренные квадратурные формулы применимы для вычисления данного интеграла, но их эффективность различна. Формула трапеций и формула прямоугольников требуют увеличения числа разбиений при повышении точности. Формула Симпсона обеспечивает наибольшую точность при минимальном количестве разбиений. Поэтому для данного интеграла наиболее эффективной оказалась составная формула Симпсона.

Вывод.

В ходе лабораторной работы были изучены составные квадратурные формулы для численного вычисления определенного интеграла. Были реализованы подпрограмма вычисления подынтегральной функции, составная формула прямоугольников, составная формула трапеций и составная формула Симпсона. Для практической оценки точности было применено правило Рунге: интеграл вычислялся на сетках с шагами h и $h/2$, после чего по разности полученных значений определялась оценка погрешности. Проведены вычисления для значений $\epsilon_{ps} = 0.01$, $\epsilon_{ps} = 0.001$ и $\epsilon_{ps} = 0.0001$, при этом количество разбиений n автоматически удваивалось до тех пор, пока оценка погрешности не становилась меньше заданной точности. По результатам вычислений установлено, что все три формулы позволяют получить значение интеграла с требуемой точностью, однако их эффективность различается. Формулы прямоугольников и трапеций при уменьшении ϵ_{ps} потребовали увеличения числа разбиений до $n = 32$, тогда как формула Симпсона во всех рассмотренных случаях достигла требуемой точности уже при $n = 4$. Это подтвердило более высокую точность формулы Симпсона и показало, что для данного интеграла она является наиболее эффективной из рассмотренных квадратурных формул.

ПРИЛОЖЕНИЕ А

Название файла: main.cpp

```
#include <iostream>
#include <cmath>
#include <iomanip>
#include <string>

using namespace std;

// 1. Подынтегральная функция для варианта №17
double f(double x) {
    return (x / (1.0 + x * x)) * exp(-x * x);
}

// 2. Метод прямоугольников
double rectangleRule(double a, double b, int n) {
    double h = (b - a) / n;
    double sum = 0.0;
    for (int i = 0; i < n; ++i) {
        double x_mid = a + i * h + h / 2.0;
        sum += f(x_mid);
    }
    return h * sum;
}

// 3. Метод трапеций
double trapezoidRule(double a, double b, int n) {
    double h = (b - a) / n;
    double sum = (f(a) + f(b)) / 2.0;
    for (int i = 1; i < n; ++i) {
        sum += f(a + i * h);
    }
    return h * sum;
}

// 4. Метод Симпсона
double simpsonRule(double a, double b, int n) {
    if (n % 2 != 0) n++;
    double h = (b - a) / n;
    double sum = f(a) + f(b);
    for (int i = 1; i < n; ++i) {
        double x_i = a + i * h;
        if (i % 2 != 0) sum += 4.0 * f(x_i);
        else sum += 2.0 * f(x_i);
    }
    return (h / 3.0) * sum;
}

void solveForEps(double a, double b, double eps) {
    cout << "\n>>> ТОЧНОСТЬ eps = " << eps << " <<<" << endl;
    cout << left << setw(18) << "Метод" << " | "
        << setw(4) << "n" << " | "
        << setw(10) << "h" << " | "
        << setw(20) << "Значение интеграла" << " | "
        << "Погрешность" << endl;
    cout << string(85, '-') << endl;
```

```

string names[] = {"Прямоугольники", "Трапеции", "Симпсон"};
int divisors[] = {3, 3, 15};

for (int k = 0; k < 3; ++k) {
    int n = 2;
    double Ih, Ih2, error;
    while (true) {
        if (k == 0) { Ih = rectangleRule(a, b, n); Ih2 =
rectangleRule(a, b, 2 * n); }
        else if (k == 1) { Ih = trapezoidRule(a, b, n); Ih2 =
trapezoidRule(a, b, 2 * n); }
        else { Ih = simpsonRule(a, b, n); Ih2 = simpsonRule(a, b, 2 *
n); }

        error = abs(Ih2 - Ih) / (double)divisors[k];

        if (error < eps) {
            int final_n = 2 * n;
            double final_h = (b - a) / final_n;

            cout << left << setw(18) << names[k] << " | "
                << setw(4) << final_n << " | "
                << fixed << setprecision(6) << setw(10) << final_h
<< " | "
                << setprecision(10) << setw(20) << Ih2 << " | "
                << scientific << setprecision(2) << error << endl;
            break;
        }
        n *= 2;
        if (n > 2000000) break;
    }
}

int main() {
    setlocale(LC_ALL, "Russian");
    double a = 0.0;
    double b = 1.0;
    double precisions[] = {0.01, 0.001, 0.0001};

    cout << "РЕЗУЛЬТАТЫ ДЛЯ ВАРИАНТА №17" << endl;
    cout << "Интервал: [0, 1]" << endl;

    for (double eps : precisions) {
        solveForEps(a, b, eps);
    }
    return 0;
}

```